

JanHelp: AI-Enabled Smart City Complaint and Resolution Platform

Team JanHelp
Bhalodiya Kartik
Sapariya Yashvi

Project Overview and Introduction

Project Title: JanHelp: AI-Enabled Smart City Complaint and Resolution Platform

Team Members: Bhalodiya Kartik and Sapariya Yashvi

University or Organization: Team JanHelp, Smart CITY Hackathon Submission

JanHelp is a civic technology platform that helps citizens report city problems and helps municipal departments receive structured, verified, and actionable complaint records. The project combines a Flutter mobile application, a Python Django REST backend, PostgreSQL-ready data storage, Cloudinary media storage, Google Gemini image analysis, multilingual AI-assisted complaint intake, duplicate detection, and automatic nearest-department assignment. It is designed for real complaints such as potholes, water leakage, garbage collection, drainage overflow, electricity faults, traffic issues, construction problems, police concerns, cyber complaints, and other public-service failures.

The project is directly relevant to the United Nations Sustainable Development Goals. It supports **SDG 11: Sustainable Cities and Communities** by improving civic service delivery and urban responsiveness. It supports **SDG 9: Industry, Innovation and Infrastructure** by applying AI, mobile access, and cloud infrastructure to public administration. It supports **SDG 16: Peace, Justice and Strong Institutions** by increasing transparency, tracking, proof-based resolution, and accountability. It also contributes to **SDG 10: Reduced Inequalities** because chatbot and voice/call complaint flows can help citizens who struggle with long digital forms, typing, or formal government language.

Problem Statement and Objectives

Most city complaint systems fail at intake. A citizen may see a pothole, broken streetlight, water leak, garbage pile, or unsafe public issue, but the official complaint process is often manual, slow, and confusing. Users must select the correct category, write a clear description, provide accurate location, upload relevant proof, and then wait without knowing whether the correct department received the case. In practice, complaints are often incomplete, wrongly categorized, duplicated, or routed to the wrong office. Departments then waste time checking proof, asking for missing details, and manually deciding ownership instead of solving the problem.

This creates three connected failures. First, citizens lose trust because reporting feels useless. Second, departments receive poor-quality records that are hard to act on. Third, city administrators lack clean data for planning and performance measurement. A simple online form does not solve this because the hard part is not only collecting text; the hard part is converting messy citizen input into a verified work order.

The main objectives of JanHelp are:

- Make civic reporting fast and accessible through mobile, guest, OTP, manual, chatbot, and voice/call-assisted flows.
- Improve complaint quality using category/subcategory selection, dynamic fields, GPS/manual location, proof upload, and AI image analysis.
- Reduce department workload through duplicate detection and automatic nearest-department assignment.

- Provide transparency through complaint ID, status tracking, resolution proof, citizen rating, and reopen flow.
- Give city admins and departments dashboards for workload, trends, pending cases, assigned cases, and service accountability.

Proposed Solution

JanHelp proposes a complete complaint lifecycle system rather than a basic reporting form. A citizen can manually fill a structured complaint, chat with an AI assistant, or use a call-style voice complaint prototype. The system collects the problem, category, subcategory, description, location, image proof, contact preference, and language. The backend validates the data, checks for duplicates, verifies proof where required, creates a complaint number, and routes the case to the nearest suitable department.

Component	Role in the solution
Flutter mobile app	Citizen-facing reporting, tracking, localization, image upload, location capture, chatbot screens, and complaint status flow.
Python Django backend	REST APIs, authentication, complaint lifecycle, category management, admin dashboards, proof handling, guest tracking, and department routing.
PostgreSQL database	Production-ready relational data model for users, complaints, departments, cities, categories, media, feedback, and reopen records.
Cloudinary media storage	Stores complaint photos, resolution proof, and media assets outside the application server.
Google Gemini AI	Analyzes image proof and supports the advanced voice/call intelligence concept.
DigitalOcean deployment target	Practical cloud target for hosting backend services, database connectivity, domain configuration, and scalable deployment.

The innovative aspect is the combination of AI intake and civic operations. Manual complaint filling is still available for users who want direct control. Chatbot complaint filling helps users describe issues naturally, then converts the conversation into structured department-ready data. AI call complaint filling is positioned as an accessibility feature: citizens select a language, speak the problem, and the system builds a complaint draft from the transcript. Image analysis checks whether uploaded proof is valid, relevant, and strong enough before it reaches staff. Duplicate detection prevents repeated tickets for the same public issue. Automatic nearest-department assignment uses complaint category, city/state scope, and distance logic to send the case to the correct operational team.

Key features include OTP and guest access, manual complaint forms, AI chatbot complaint creation, voice/call prototype logic, multilingual support, GPS/manual location capture, image proof verification, duplicate detection, automatic department assignment, complaint tracking, resolution proof, citizen rating, reopen workflow, city admin dashboard, and department dashboard. The main challenges addressed are poor complaint quality, fake or weak proof, duplicate reports, wrong department routing, lack of citizen trust, and lack of city-level analytics.

Results and Feasibility

JanHelp has high practical feasibility because the repository already contains a full-stack implementation: a Django backend, Flutter mobile app, REST API layer, complaint models, serializers, authentication flow, category and department handling, proof verification logic, dashboards, deployment configuration, and submission assets. The prototype can be demonstrated through one complete journey: a citizen reports an issue, uploads proof, AI validates the image, duplicate detection runs, the complaint is assigned to the nearest department, the department updates status, uploads resolution proof, and the citizen rates or reopens the case.

The potential impact is significant. Citizens get a simpler reporting experience and visible accountability. Departments get cleaner, more complete, and less duplicated complaint records. City

admins get data for identifying hotspots, overloaded departments, unresolved cases, and recurring infrastructure failures. Over time, the system can shift civic service delivery from reactive manual handling to data-driven operations.

Future development should focus on production hardening. Exposed client-side API keys must be rotated and moved server-side. The voice/call module should be fully wired into the app before being claimed as live-ready. Spatial indexing can improve duplicate checks at larger city scale. Notifications, analytics, ward-level heatmaps, SLA escalation, officer assignment, and DigitalOcean production deployment can make the system stronger for real city use.

Support and partnerships: No formal government or institutional partnership is claimed in this report. The project uses open-source and cloud technologies including Flutter, Django, PostgreSQL, Cloudinary, and Google Gemini.

References and sources: Project source code in the Smart CITY repository; United Nations Sustainable Development Goals; official Flutter, Django REST Framework, PostgreSQL, Cloudinary, DigitalOcean, and Google Gemini documentation.